



고려대학교
KOREA UNIVERSITY

신뢰할 수 있는 APR을 위하여

소프트웨어 분석 연구실
송도원

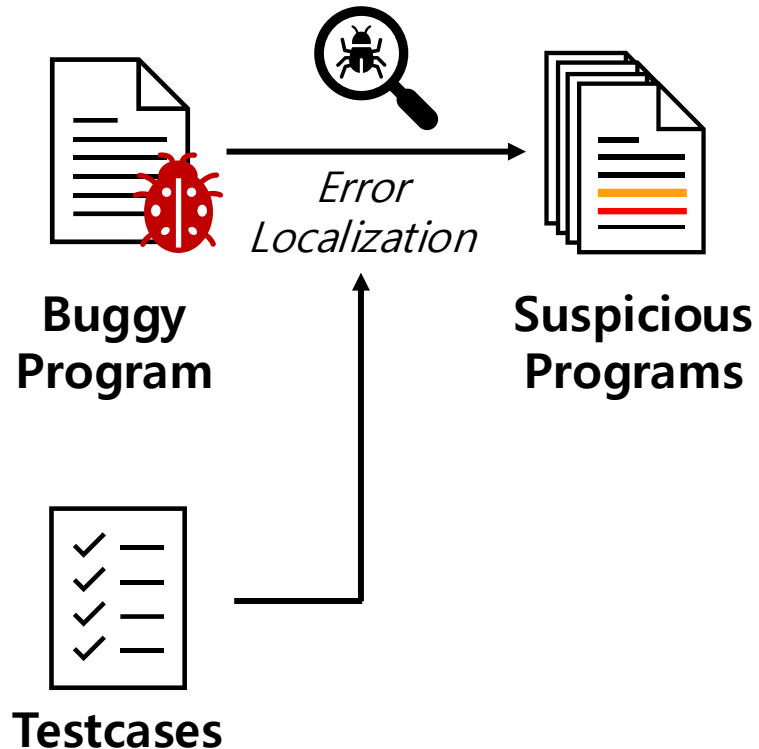


2026.07.14

소프트웨어 재난 연구센터 여름 워크숍 @ 경북대학교

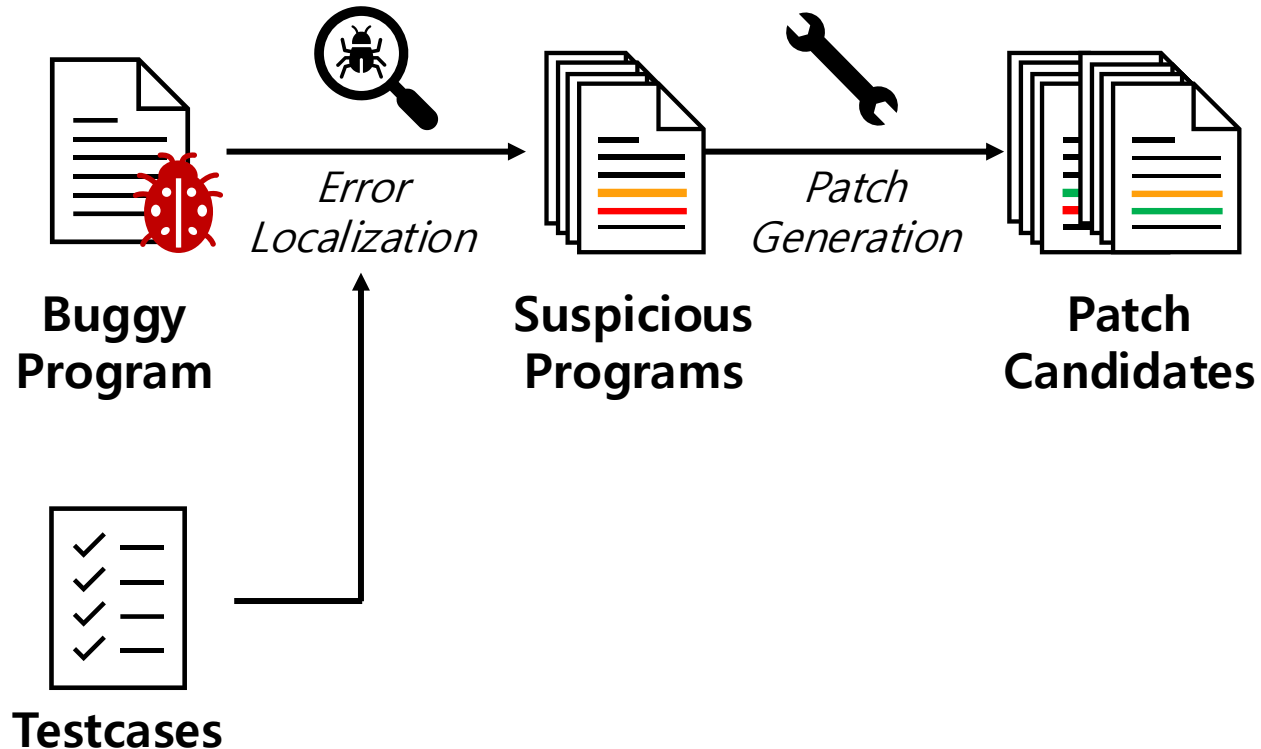
프로그램 자동 수정 (APR)

- 목표: 소프트웨어의 버그를 **자동으로 수정하여** 디버깅에 드는 비용을 줄이자



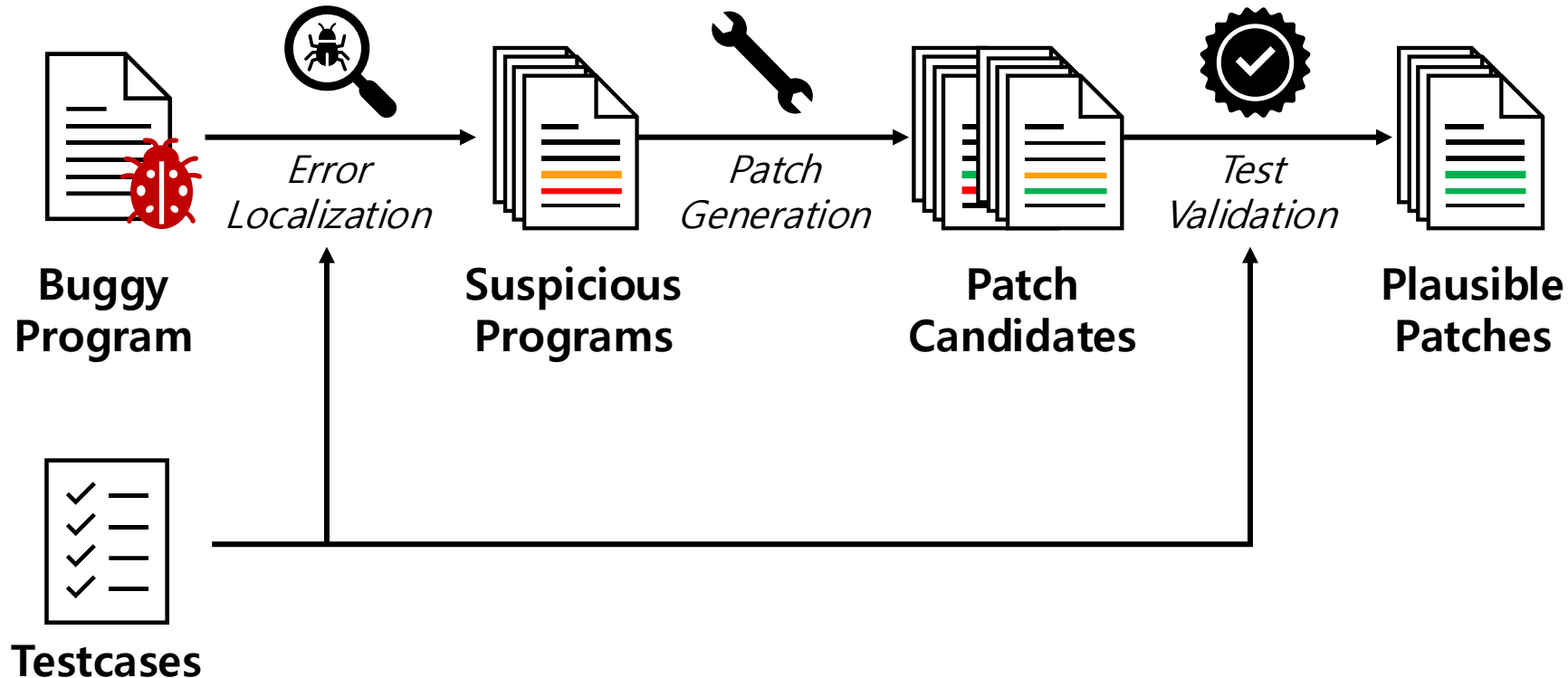
프로그램 자동 수정 (APR)

- 목표: 소프트웨어의 버그를 **자동으로 수정하여** 디버깅에 드는 비용을 줄이자



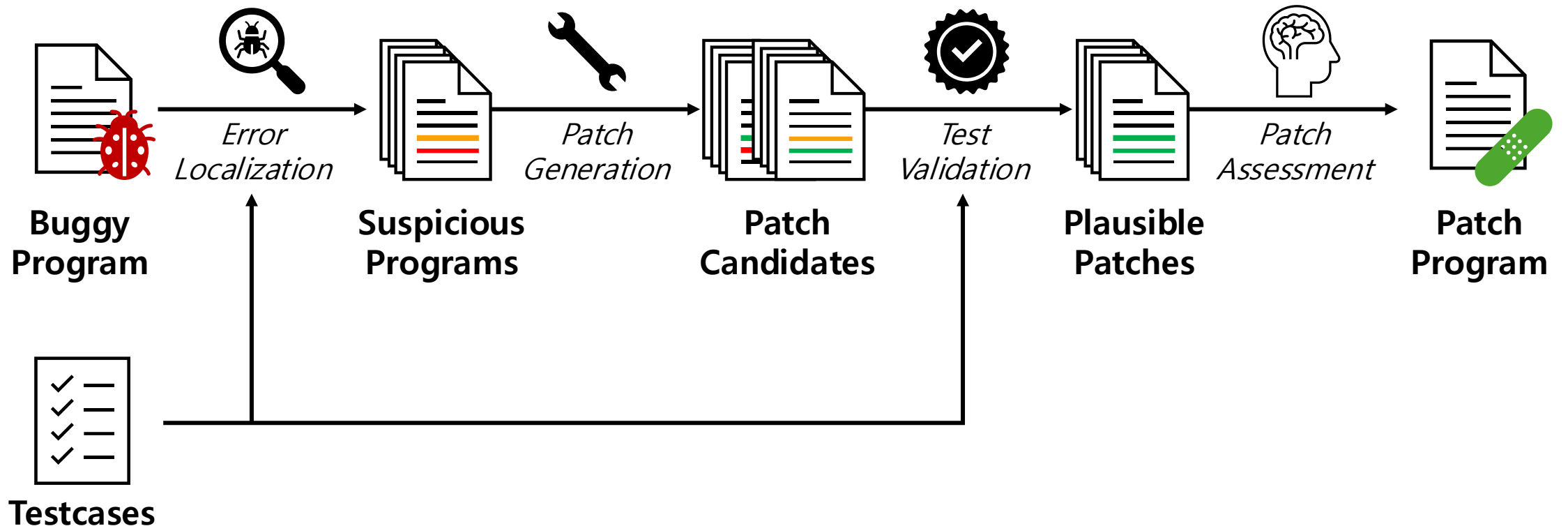
프로그램 자동 수정 (APR)

- 목표: 소프트웨어의 버그를 **자동으로 수정하여** 디버깅에 드는 비용을 줄이자



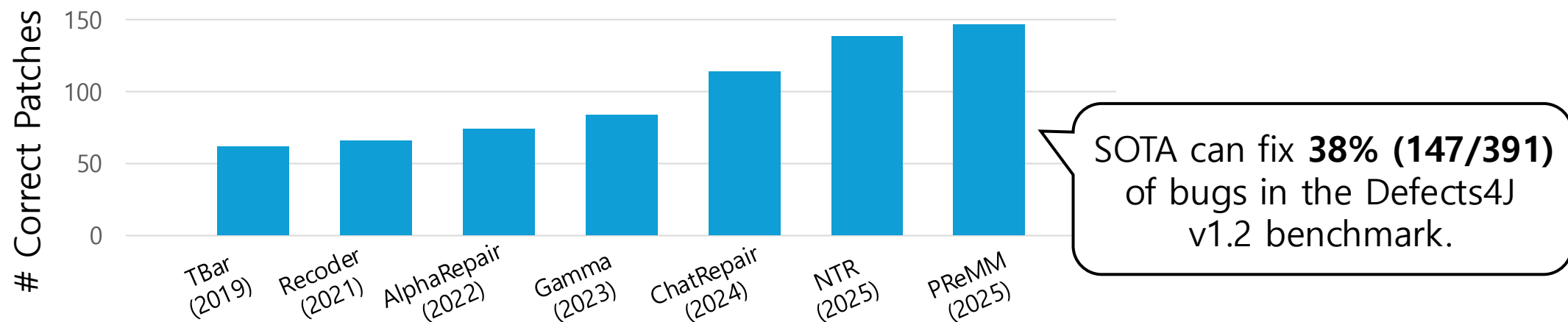
프로그램 자동 수정 (APR)

- 목표: 소프트웨어의 버그를 **자동으로 수정하여** 디버깅에 드는 비용을 줄이자



APR 기술의 현재

- LLM발전과 함께, 최근 APR의 성능이 굉장히 빠르게 향상 중



- APR의 실용성: 코딩 에이전트



Claude



APR의 이상과 현실

- 일은 APR이 하고 사람은 놀기



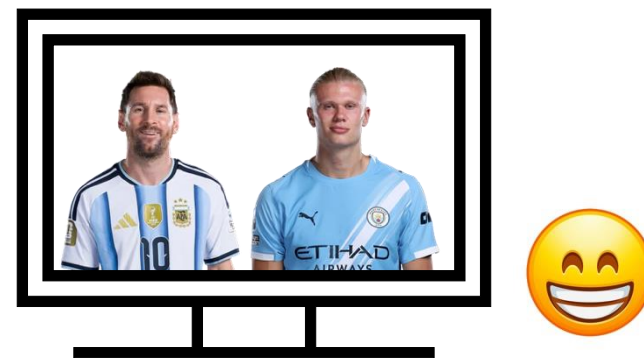
APR의 이상과 현실

- 일은 APR이 하고 사람은 놀기

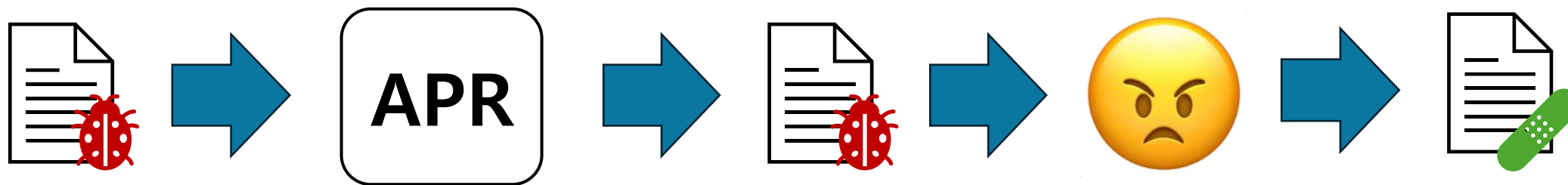


APR의 이상과 현실

- 일은 APR이 하고 사람은 놀기

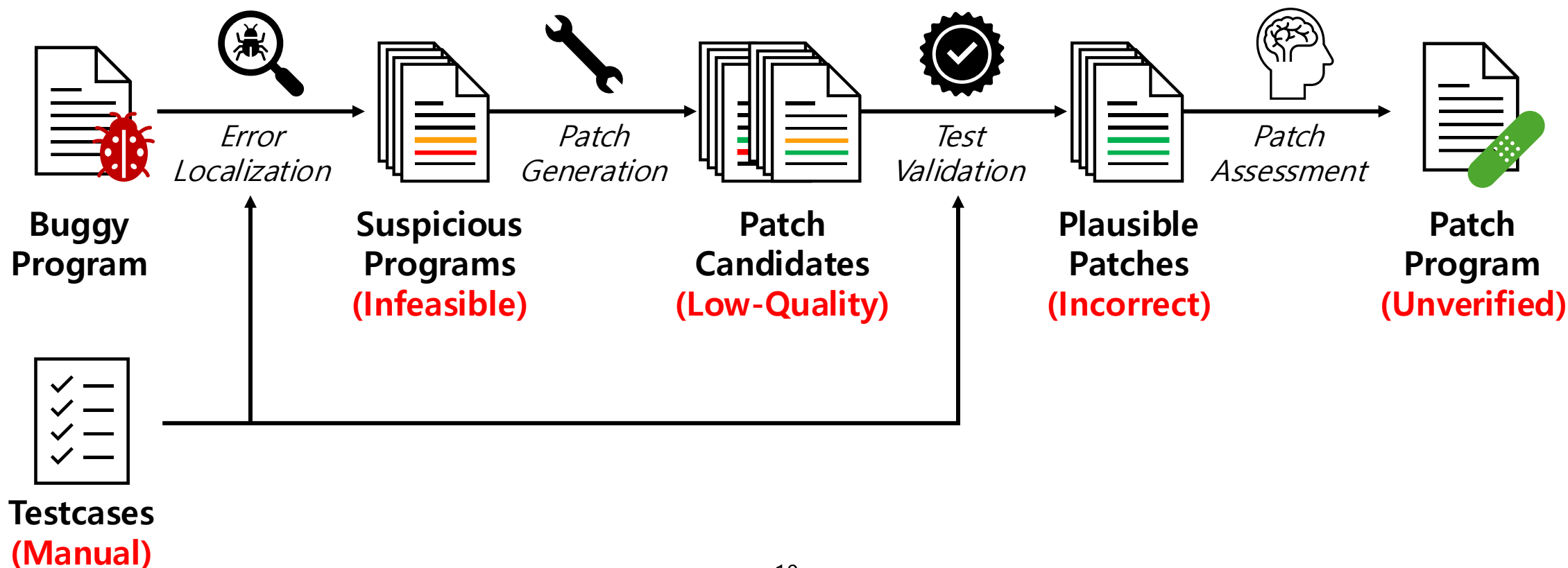


- 생성된 코드를 믿을 수가 없음



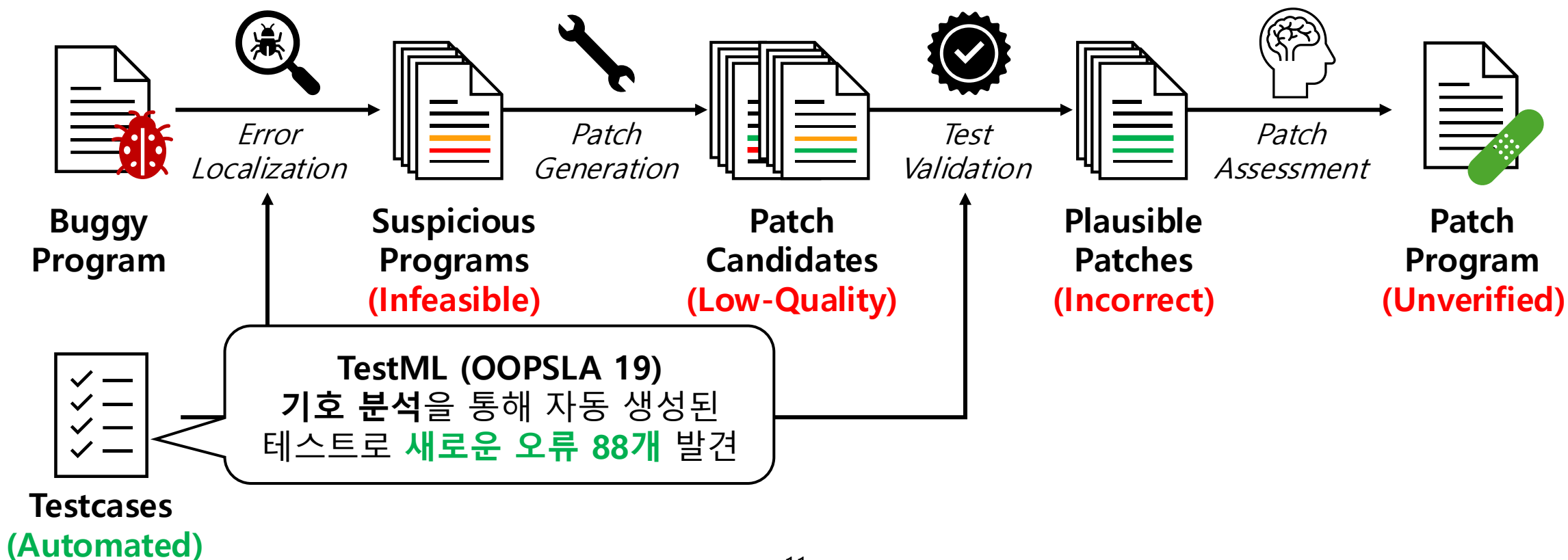
문제: 신뢰할 수 없는 APR

- APR이 각 요소의 한계로 인하여 **틀린 패치**를 생성하고, 이는 사용자가 **APR 기술을 신뢰할 수 없게함**



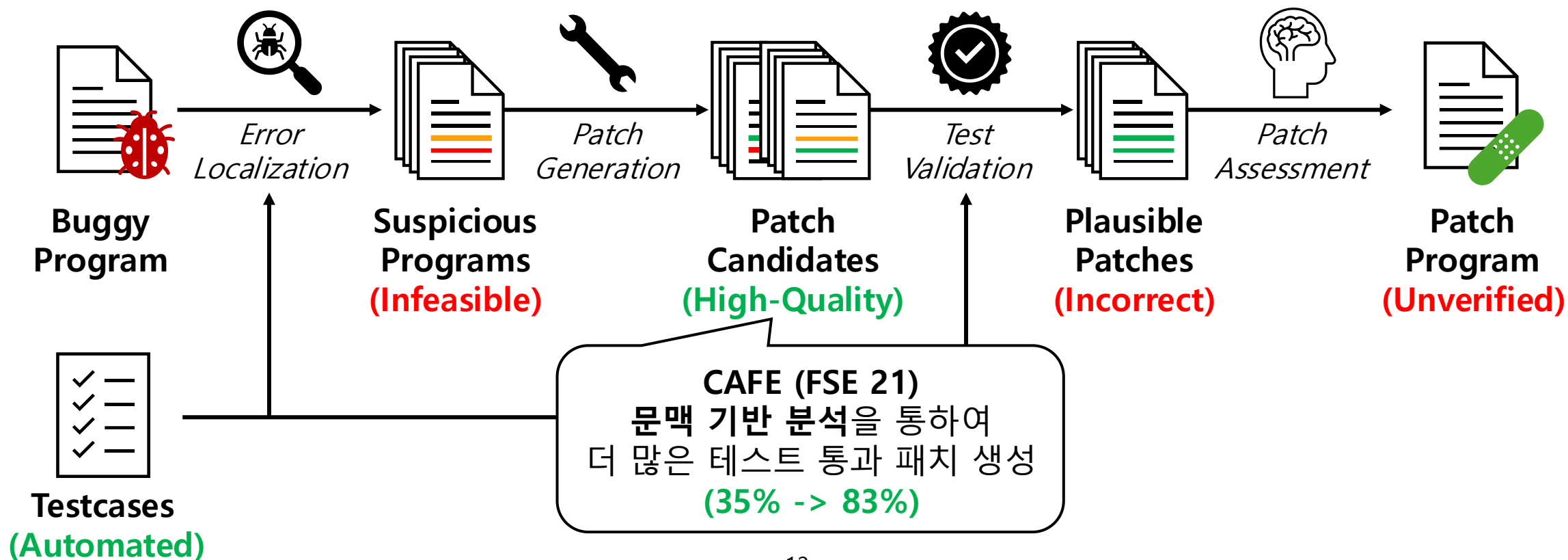
목표: APR을 믿을 수 있게 만들기

- 정적분석을 바탕으로 APR의 각 요소를 개선해 신뢰성 있는 APR을 구현



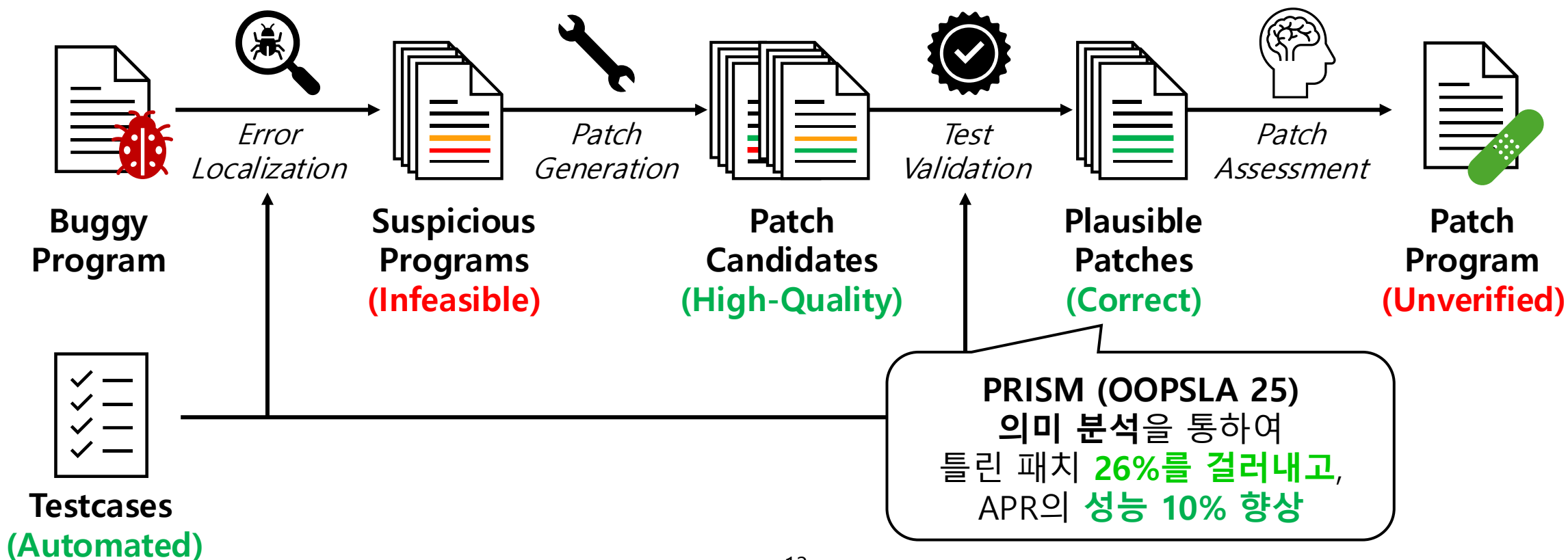
목표: APR을 믿을 수 있게 만들기

- 정적분석을 바탕으로 APR의 각 요소를 개선해 신뢰성 있는 APR을 구현



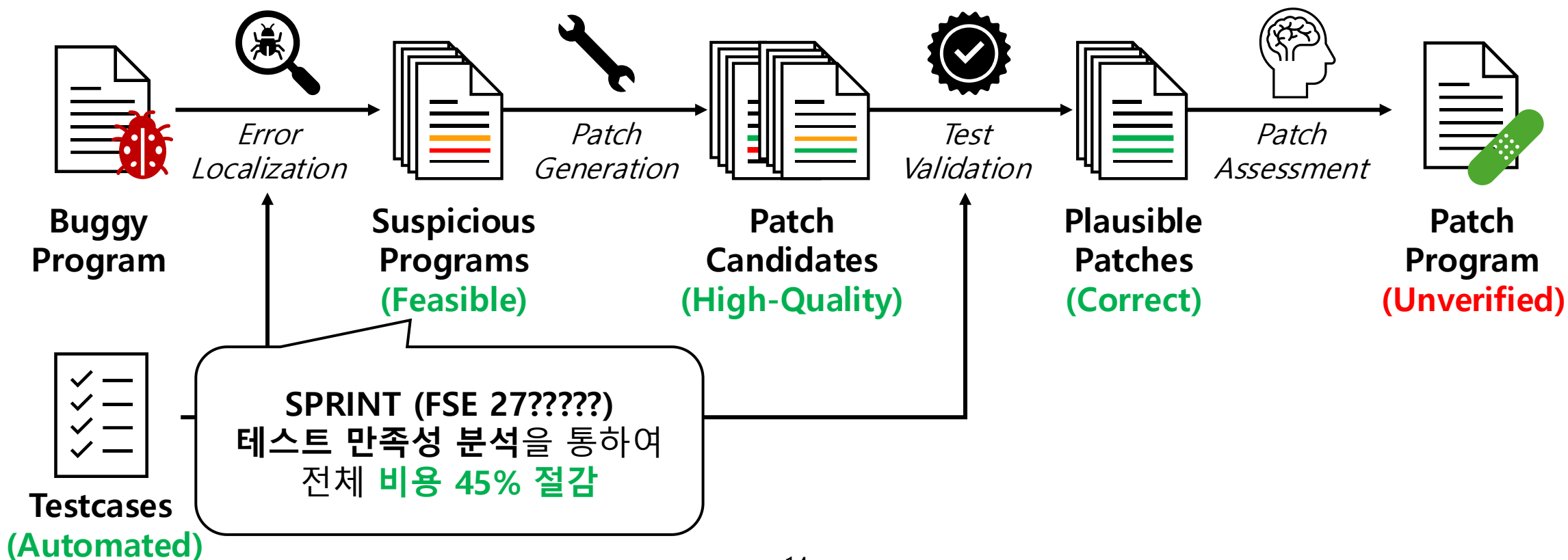
목표: APR을 믿을 수 있게 만들기

- 정적분석을 바탕으로 APR의 각 요소를 개선해 신뢰성 있는 APR을 구현



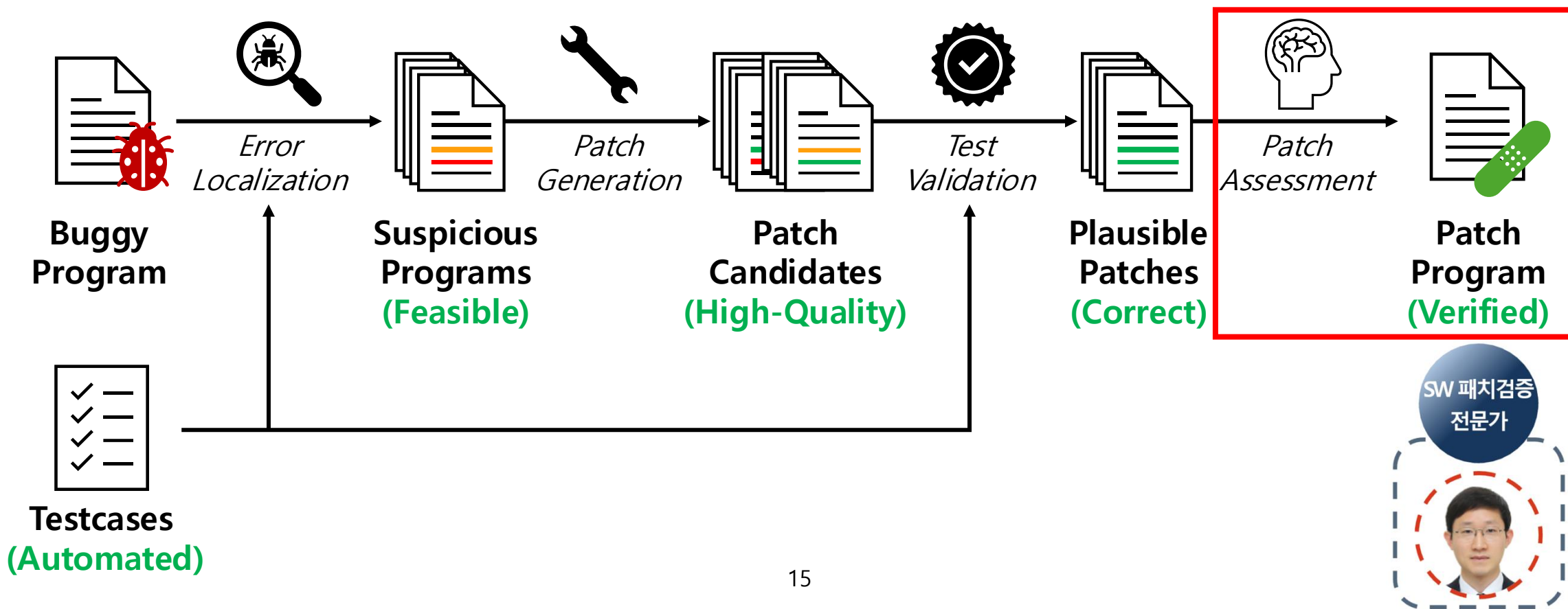
목표: APR을 믿을 수 있게 만들기

- 정적분석을 바탕으로 APR의 각 요소를 개선해 신뢰성 있는 APR을 구현

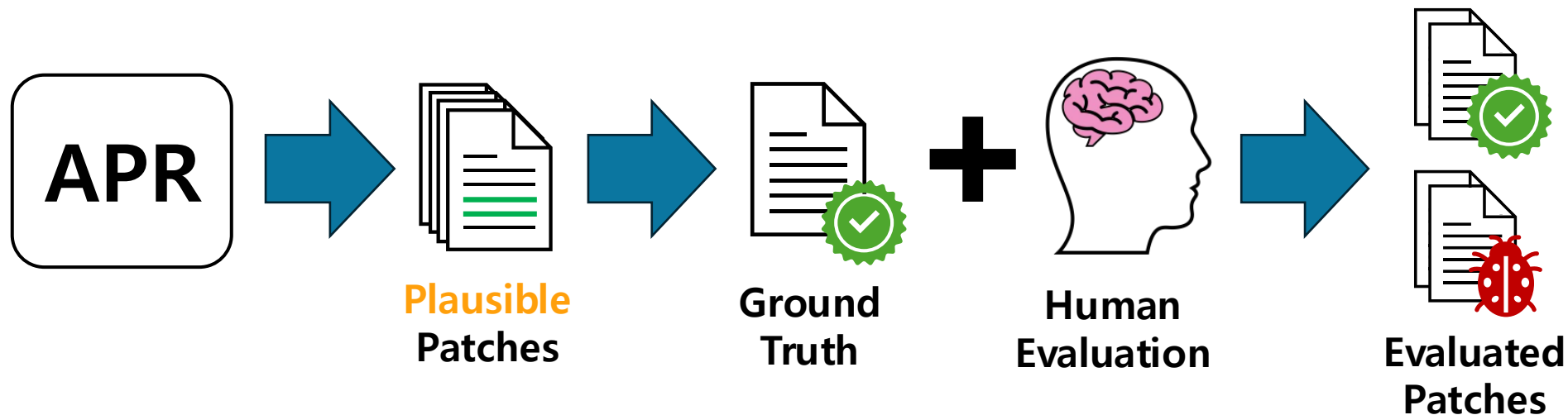


목표: APR을 믿을 수 있게 만들기

- 정적분석을 바탕으로 APR의 각 요소를 개선해 신뢰성 있는 APR을 구현



APR 연구의 병목: 패치 정답 평가



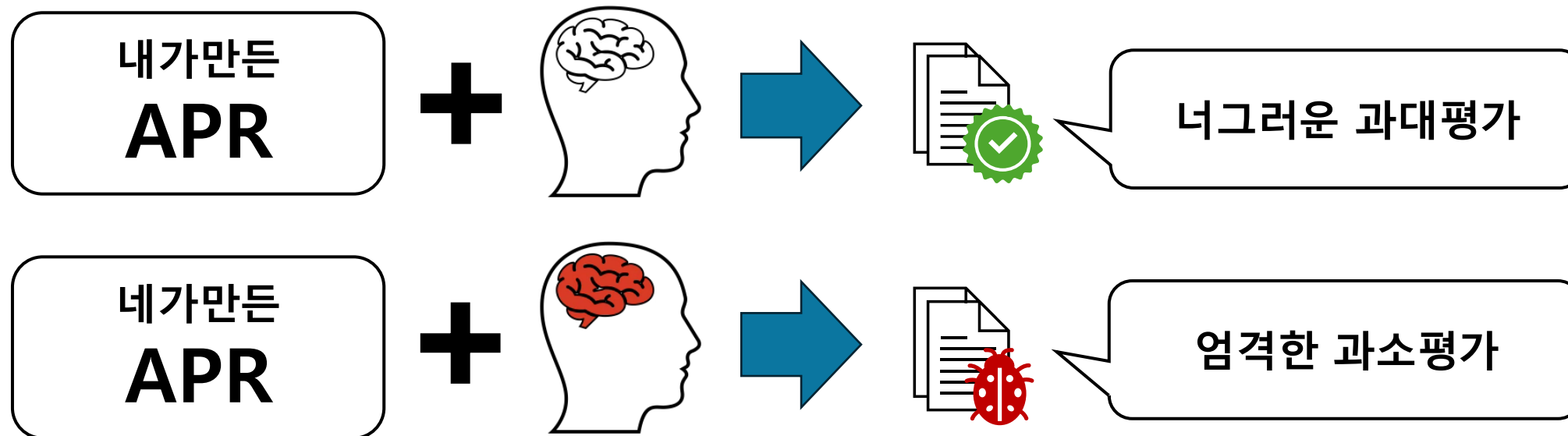
- 성능 평가에 가장 중요한 부분이 주관적인 수동 평가에 기반
 - *"authors of APR techniques annotate correctness labels of patches generated by their and competing tools by themselves"* - ICSE 19
- 생성되는 패치의 수가 굉장히 많아 오래 걸리고 실수도 잦음
 - *"Manual annotation by authors is prone to human biases and requires repetitive and expensive manual effort."* - TSE 23

APR 평가의 현실

- 누가 평가하냐에 따라 같은 기술도 평가가 달라짐
 - TBar (ISSTA 17): 몇년 후 성능 과대평가 지적 (43 => 27)
 - Recoder (FSE19): 몇년 후 다른 논문에서 성능 하락 리포트 (53 => 45)
- 과소, 과대평가된 성능으로 후속 연구에도 영향을 줌

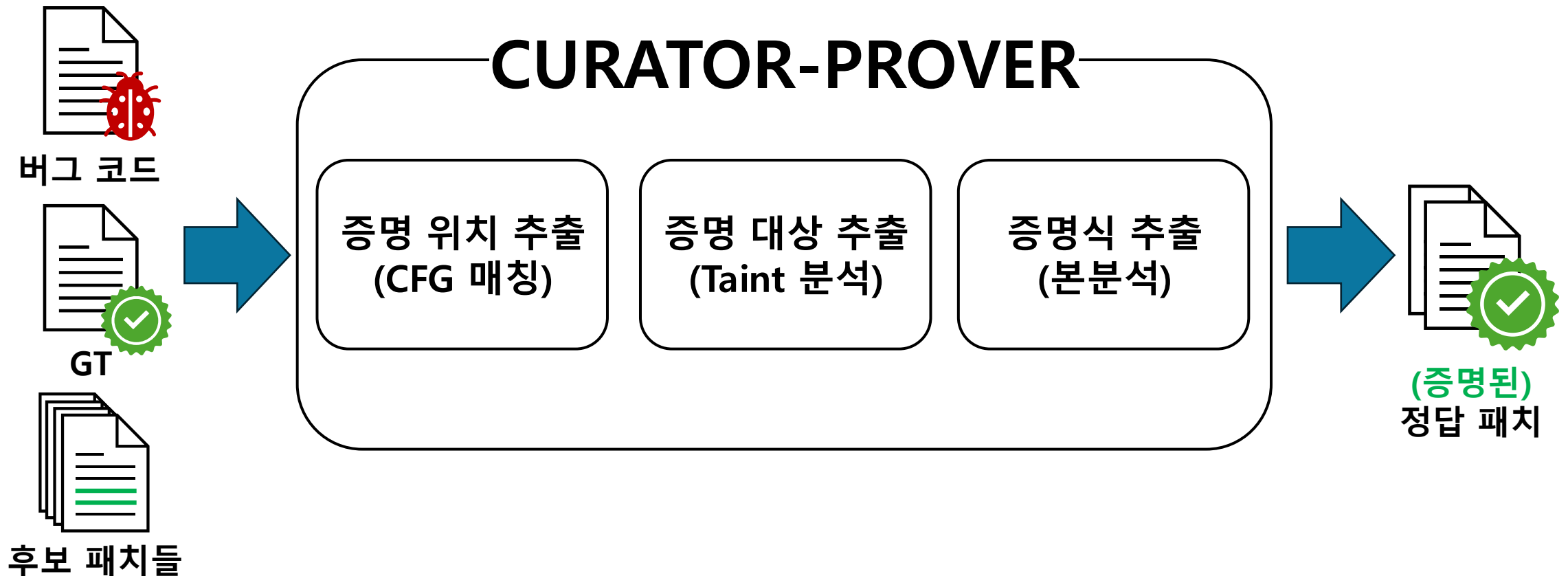
APR 평가의 현실

- 누가 평가하냐에 따라 같은 기술도 평가가 달라짐
 - TBar (ISSTA 17): 몇년 후 성능 과대평가 지적 (43 => 27)
 - Recoder (FSE19): 몇년 후 다른 논문에서 성능 하락 리포트 (53 => 45)
- 과소, 과대평가된 성능으로 후속 연구에도 영향을 줌



CURATOR: 부분 증명 기반 자동 평가

- Ground Truth와 동작이 같은지 검증하여 패치 자동 평가
 - 증명이 성공하면 해당 패치가 진짜 정답 패치임을 보장



Key Idea: 부분 증명으로 쉽게 증명하기

- 두 패치가 아주 국소적인 수정을 하고, 원본의 대부분을 공유하기에 전체 증명이 필요 없음
- 예제 벤치마크 (Chart-13): Math.max의 두 인자가 같기 때문에 두 패치는 동치

```
RectConst c4 =  
- new RectConst(0.0,new Range(0.0, constraint.getWidth() - w[2]), ...);  
+ new RectConst(0.0,new Range(0.0, Math.max(constraint.getWidth() - w[2], 0.0)), ...);
```

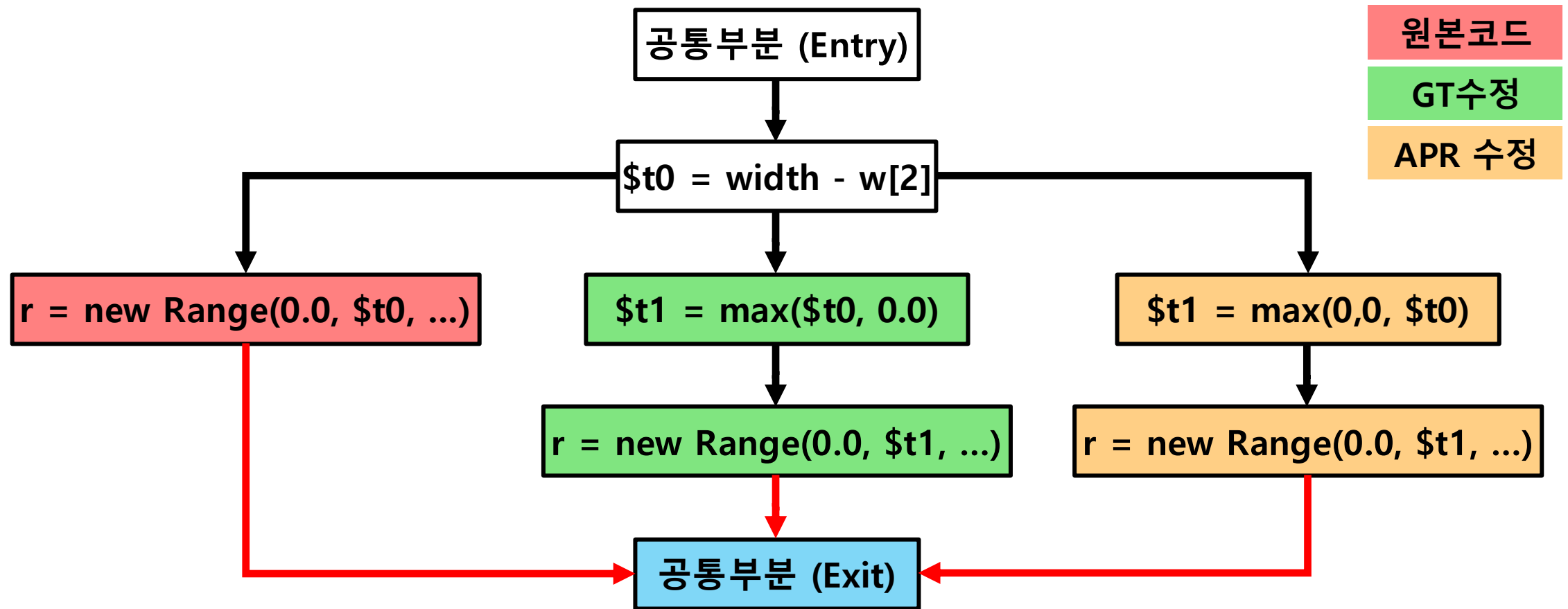
개발자 패치

```
RectConst c4 =  
- new RectConst(0.0,new Range(0.0, constraint.getWidth() - w[2]), ...);  
+ new RectConst(0.0,new Range(0.0, Math.max(0.0, constraint.getWidth() - w[2])), ...);
```

APR 패치

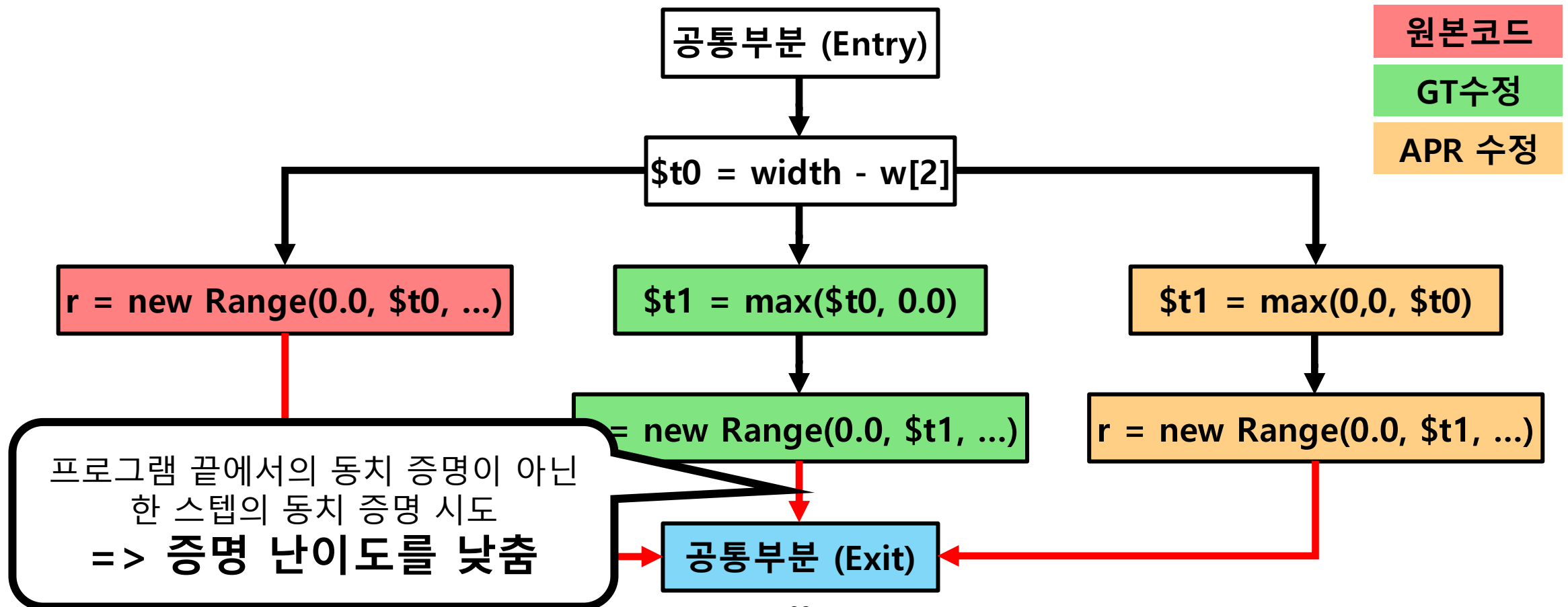
Step1: 증명 위치 추출

- 패치가 처음으로 영향을 주는 지점에서만 증명을 시도



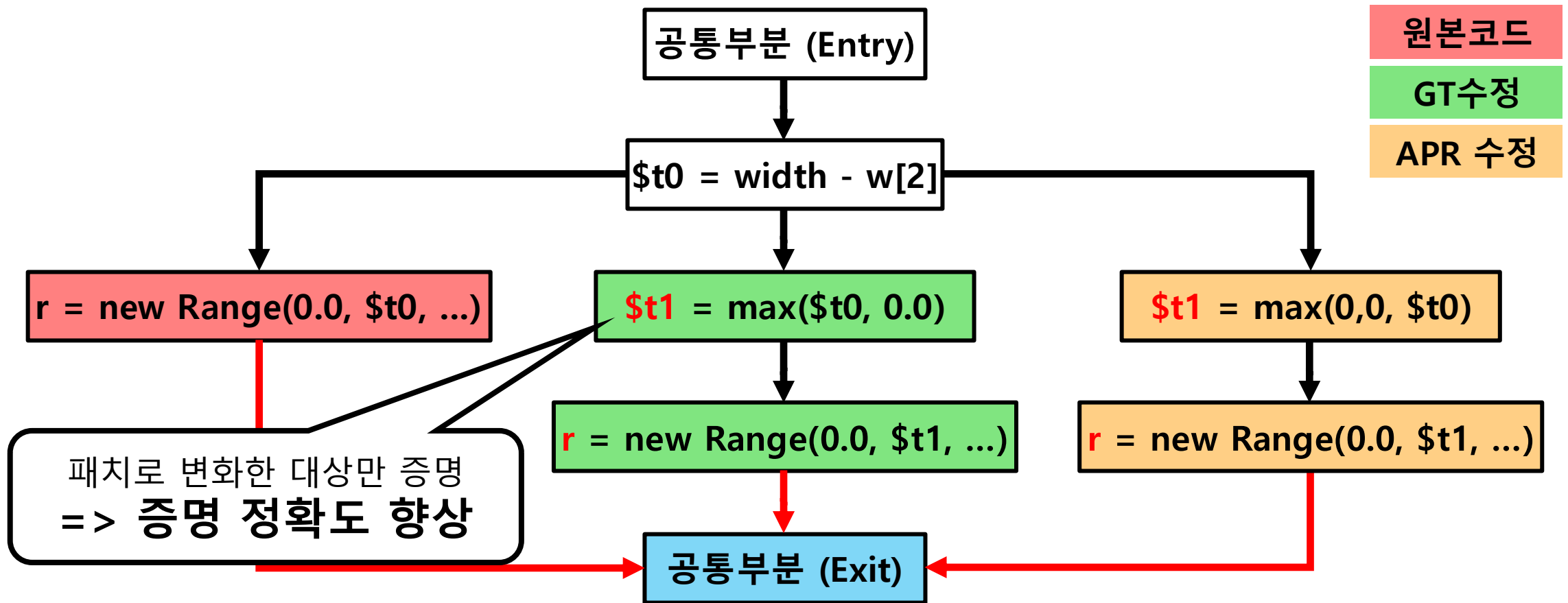
Step1: 증명 위치 추출

- 패치가 처음으로 영향을 주는 지점에서만 증명을 시도



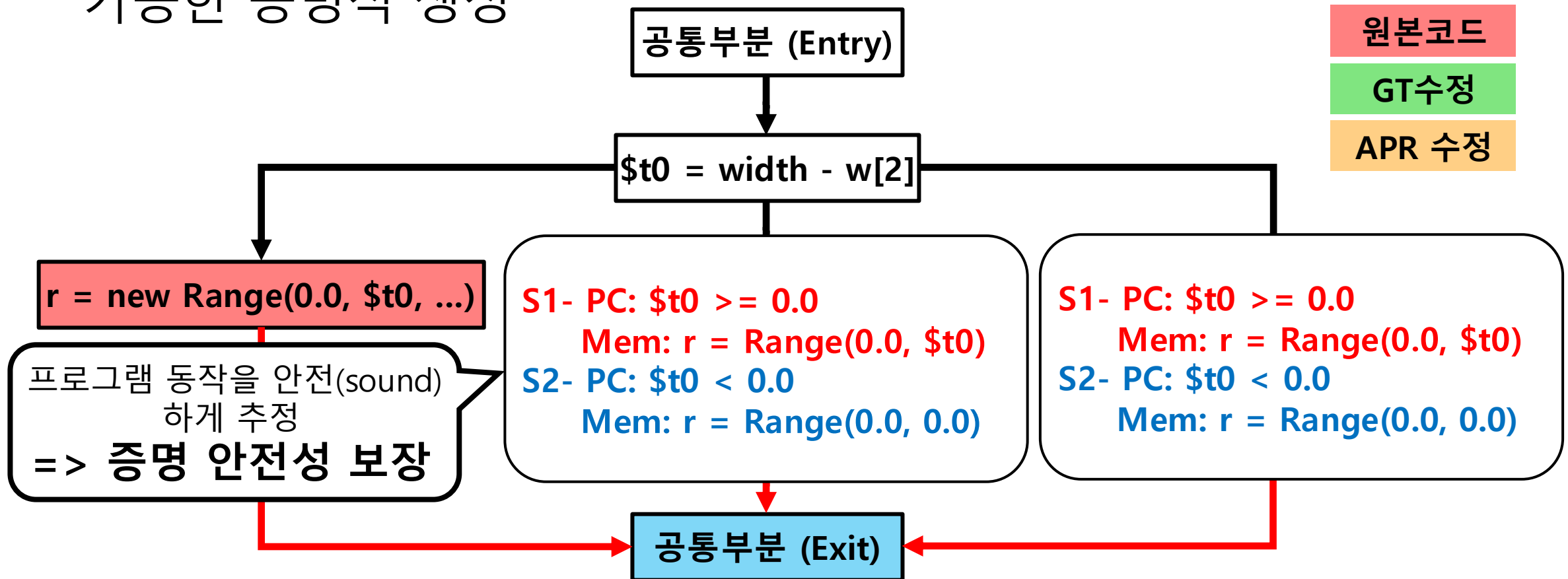
Step2: 증명 대상 추출

- 메모리 전체 동치가 아닌, 패치가 영향을 준 대상만 동치 증명



Step3: 증명식 추출

- 프로그램 행동을 안전하게 추정 (Over-approximation)하여 보장 가능한 증명식 생성



초기 실험 결과

- 벤치마크: 기존 연구 (Song et al.) 에서 동일 기준으로 평가한 2000개 이상의 패치 집합
- 현재까지 증명 시도한 **92개의 정답 패치중 82% 증명 성공**
- **15개의 잘못된 저자 평가 (~~by Dowon Song~~) 발견**
 - 실제로는 정답 패치인데, 보수적으로 오답 패치로 평가
 - 실제 APR기술의 성능을 과소 평가함

결론

- **My Long-term Goal:** 신뢰 가능한 APR을 만들어 사람들이 놀 수 있는 시간을 만들자
 - 정적 분석을 활용하여 APR의 각 한계를 극복하자
- **Current Goal:** 패치 자동 검증을 통해 주관적인 APR 평가부터 신뢰할 수 있게 만들자
- **Approach:** 패치 변화만 집중해 효과적이고 쉽게 검증 하자
- **Evaluation:** 벤치마크 82% 검증 성공, 15개의 사람 오류 발견